

Exercise 7: Multivariate Optimization 2

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8
```

R

```
install.packages("remotes")  
remotes::install_version("ggplot2", "3.5.1")
```

Setup

A radial basis function (RBF) network has been fitted to a black-box function, giving the model

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}, \mathbf{x} \mapsto \sum_{i=1}^2 w_i \cdot \rho(\|\mathbf{x} - \mathbf{c}_i\|_{S_i})$$

with $\mathbf{c}_1 = (-1.1, 1.1)^\top$, $\mathbf{c}_2 = (0.8, -0.8)^\top$, weights $w_1 = 1$, $w_2 = -1$, **biweight (quartic) kernel**

$$\rho : \mathbb{R} \rightarrow \mathbb{R}, u \mapsto \begin{cases} (1 - u^2)^2 & |u| < 1 \\ 0 & \text{otherwise} \end{cases}$$

and Mahalanobis distances $\|\cdot\|_{S_i}$ with covariance matrices $S_1 = \mathbf{I}$, $S_2 = \begin{pmatrix} 1.1 & -0.9 \\ -0.9 & 1.1 \end{pmatrix}$. The Mahalanobis distance is $\|\mathbf{x} - \mathbf{c}\|_S = \sqrt{(\mathbf{x} - \mathbf{c})^\top S^{-1}(\mathbf{x} - \mathbf{c})}$.

Note: We chose this kernel and distance for educational purposes; in practice, Gaussian kernels and Euclidean distances are more common.

(a) Surface plot

Plot f on the range $[-2, 2] \times [-2, 2]$.

Solution.

R

```
library(ggplot2)

c1 <- c(-1.1, 1.1)
c2 <- c(0.8, -0.8)
S2 <- matrix(c(1.1, -0.9, -0.9, 1.1), nrow = 2)
S2_inv <- solve(S2)

rho <- function(u) ifelse(abs(u) < 1, (1 - u^2)^2, 0)
dist1 <- function(x) sqrt(sum((x - c1)^2)) # Euclidean (S1 = I)
dist2 <- function(x) sqrt((x - c2) %*% S2_inv %*% (x - c2)) # Mahalanobis

# w_1 = +1, w_2 = -1
f <- function(x) rho(dist1(x)) - rho(dist2(x))

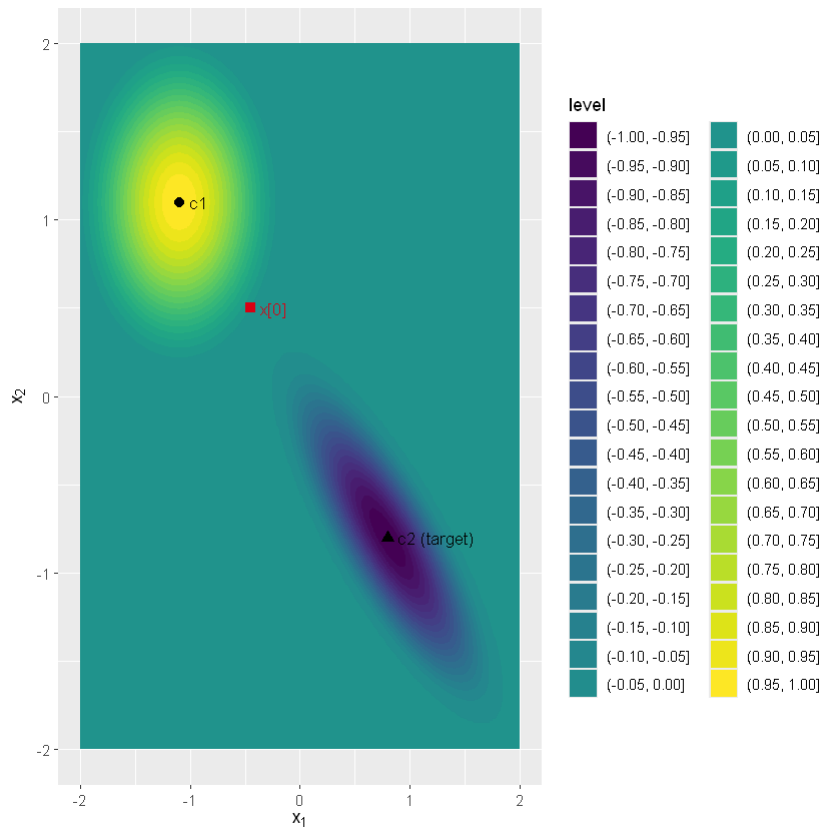
# Surface plot on [-2, 2] x [-2, 2]
x <- seq(-2, 2, by = 0.01)
xx <- expand.grid(X1 = x, X2 = x)
fxx <- apply(xx, 1, f)
df_grid <- data.frame(xx, fxx = fxx)

cont_plot <- ggplot(df_grid, aes(x = X1, y = X2, z = fxx)) +
  geom_contour_filled(binwidth = 0.05) +
  xlab(expression(x[1])) +
  ylab(expression(x[2])) +
  # landmarks: positive-bump center, negative-bump center (target), start point
  annotate("point", x = c1[1], y = c1[2], color = "black", size = 2.5) +
  annotate("text", x = c1[1], y = c1[2], label = " c1", hjust = 0,
```

```

        color = "black", size = 3.5) +
    annotate("point", x = c2[1], y = c2[2], color = "black",
            size = 2.5, shape = 17) +
    annotate("text", x = c2[1], y = c2[2], label = "  c2 (target)", hjust = 0,
            color = "black", size = 3.5) +
    annotate("point", x = -0.45, y = 0.5, color = "#D90012",
            size = 2.5, shape = 15) +
    annotate("text", x = -0.45, y = 0.5, label = "  x[0]", hjust = 0,
            color = "#D90012", size = 3.5)
cont_plot

```



Python

```

import numpy as np
import matplotlib.pyplot as plt

```

```

c1 = np.array([-1.1, 1.1])
c2 = np.array([0.8, -0.8])
S2 = np.array([[1.1, -0.9], [-0.9, 1.1]])
S2_inv = np.linalg.inv(S2)

def rho(u):
    """Biweight (quartic) kernel:  $(1 - u^2)^2$  for  $|u| < 1$ , else 0."""
    return (1 - u ** 2) ** 2 if abs(u) < 1 else 0.0

def dist1(x):
    return float(np.sqrt((x - c1) @ (x - c1))) # Euclidean (S1 = I)

def dist2(x):
    return float(np.sqrt((x - c2) @ S2_inv @ (x - c2))) # Mahalanobis

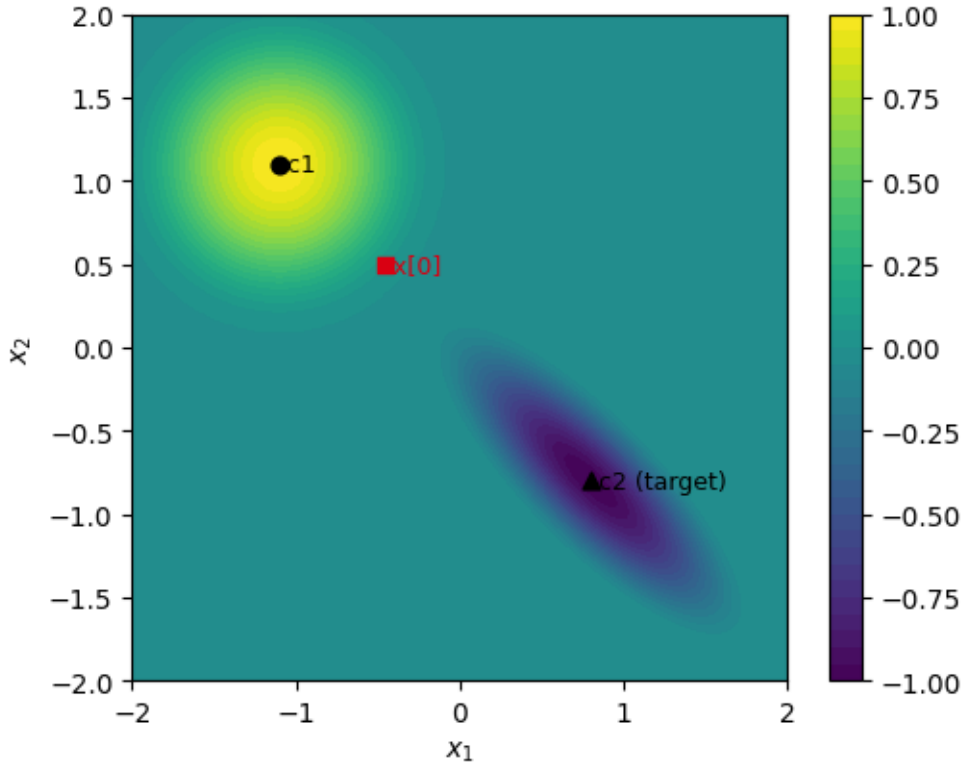
def f(x):
    """w_1 = +1, w_2 = -1."""
    x = np.asarray(x, dtype=float)
    return rho(dist1(x)) - rho(dist2(x))

# Surface plot on [-2, 2] x [-2, 2]
grid = np.arange(-2, 2 + 0.01, 0.01)
T1, T2 = np.meshgrid(grid, grid)
Z = np.array([[f(np.array([t1, t2])) for t1 in grid] for t2 in grid])

def plot_landscape(ax):
    """Render the contour with c_1, c_2, x[0] landmarks (reused by d, f)."""
    cs = ax.contourf(T1, T2, Z, levels=np.arange(-1.0, 1.0 + 0.05, 0.05))
    ax.scatter(*c1, color="black", s=40, marker="o", zorder=3)
    ax.annotate(" c1", c1, color="black", fontsize=9, va="center")
    ax.scatter(*c2, color="black", s=40, marker="^", zorder=3)
    ax.annotate(" c2 (target)", c2, color="black", fontsize=9, va="center")
    ax.scatter(-0.45, 0.5, color="#D90012", s=40, marker="s", zorder=3)
    ax.annotate(" x[0]", (-0.45, 0.5), color="#D90012", fontsize=9, va="center")
    ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
    return cs

```

```
fig, ax = plt.subplots(figsize=(5.5, 4.5))
cs = plot_landscape(ax)
fig.colorbar(cs)
plt.show()
```



The plot shows two basins: a positive bump at $\mathbf{c}_1$ (where $f = +1$) and a negative dip at $\mathbf{c}_2$ (where $f = -1$). The starting point $\mathbf{x}^{[0]} = (-0.45, 0.5)$ lies inside $\mathbf{c}_1$'s support.

Outside both supports the function is identically zero — the **dead zone** (the uniform color filling most of the plot). This flat region is why vanilla GD gets stuck in (d): a constant function has zero gradient, so any step landing in the dead zone cannot move.

(b) Disjoint supports

Show that $\bigcap_{i=1}^2 \{\mathbf{x} \in \mathbb{R}^2 \mid \rho(\|\mathbf{x} - \mathbf{c}_i\|_{S_i}) \neq 0\} = \emptyset$.

Solution.

First we analyze $\rho(u)$ for $|u| < 1$:

$$(1 - u^2)^2 = 0 \iff (1 - u^2) = 0 \iff u^2 = 1,$$

so $\rho(u) \neq 0$ for $u^2 < 1$ and $\rho(u) = 0$ for $u^2 \geq 1$. We can check this condition for both squared distances around the centers $\mathbf{c}_1, \mathbf{c}_2$:

1. $\|\mathbf{x} - \mathbf{c}_1\|_{S_1}^2 < 1 \iff \|\mathbf{x} - \mathbf{c}_1\|_2^2 < 1$ (unit circle around $\mathbf{c}_1$).
2. $\|\mathbf{x} - \mathbf{c}_2\|_{S_2}^2 = (\mathbf{x} - \mathbf{c}_2)^\top \begin{pmatrix} 1.1 & -0.9 \\ -0.9 & 1.1 \end{pmatrix}^{-1} (\mathbf{x} - \mathbf{c}_2) < 1$ (ellipse around $\mathbf{c}_2$).

To find the smallest enclosing circle of the ellipse, use the eigendecomposition of S_2 :

$$\begin{aligned} \det(S_2 - \lambda \mathbf{I}) = 0 &\iff \det \begin{pmatrix} 1.1 - \lambda & -0.9 \\ -0.9 & 1.1 - \lambda \end{pmatrix} = 0 \\ &\iff \lambda^2 - 2.2\lambda + 0.4 = 0 \iff \lambda_1 = 2.0, \lambda_2 = 0.2. \end{aligned}$$

The eigenvalues of S_2^{-1} are $1/\lambda_1 = 0.5$ and $1/\lambda_2 = 5$. Rotating into the eigenbasis $\mathbf{y} = \mathbf{V}(\mathbf{x} - \mathbf{c}_2)$ (with $|\det \mathbf{V}| = 1$) diagonalizes the quadratic form:

$$\|\mathbf{x} - \mathbf{c}_2\|_{S_2}^2 < 1 \iff 5y_1^2 + 0.5y_2^2 < 1.$$

This is an ellipse with semi-axes $1/\sqrt{5} \approx 0.45$ and $1/\sqrt{0.5} = \sqrt{2}$ (set one $y_j = 0$ and solve, e.g. $5y_1^2 = 1 \Rightarrow y_1 = 1/\sqrt{5}$). A circle centered at $\mathbf{c}_2$ enclosing it must reach the **larger** semi-axis, so radius $\sqrt{2}$ suffices.

Comparing the distance between centers with the sum of enclosing radii:

$$\|\mathbf{c}_2 - \mathbf{c}_1\|_2 = \sqrt{2 \cdot 1.9^2} \approx 2.69 > 1 + \sqrt{2} \approx 2.41,$$

so the circles cannot intersect, hence neither can the supports. Only $\rho(\|\mathbf{x} - \mathbf{c}_1\|_{S_1})$ or $\rho(\|\mathbf{x} - \mathbf{c}_2\|_{S_2})$ can be non-zero for a given $\mathbf{x} \in \mathbb{R}^2$.

(c) Global minimum, analytically

Find the global minimum of f analytically.

Hint: recall (b).

Solution.

By (b), the two RBF terms are independent — at any $\mathbf{x}$ at most one is nonzero. Combined with $\rho \geq 0$, $w_1 = +1$, and $w_2 = -1$:

- inside $\mathbf{c}_1$'s support: $f = +\rho(\cdot) \geq 0$
- inside $\mathbf{c}_2$'s support: $f = -\rho(\cdot) \leq 0$
- elsewhere: $f = 0$.

So the global minimum lives in $\mathbf{c}_2$'s support. Setting $\nabla_{\mathbf{x}}\rho(\|\mathbf{x} - \mathbf{c}_2\|_{S_2}) = \mathbf{0}$:

$$\nabla_{\mathbf{x}}\rho(\|\mathbf{x} - \mathbf{c}_2\|_{S_2}) = 2(1 - \|\mathbf{x} - \mathbf{c}_2\|_{S_2}^2) \cdot (-2)(\mathbf{x} - \mathbf{c}_2)^\top S_2^{-1} = \mathbf{0}.$$

Either $\|\mathbf{x} - \mathbf{c}_2\|_{S_2} = 1$ (the boundary, where $\rho = 0$) or $\mathbf{x} = \mathbf{c}_2$. Since $-\rho(\mathbf{0}) = -1 < 0 = -\rho(1)$, the global minimum is at $\mathbf{x} = \mathbf{c}_2$ with value $f(\mathbf{c}_2) = -1$.

(d) Two GD steps numerically

Write an R/Python script that performs two gradient descent steps starting at $\mathbf{x}^{[0]} = (-0.45, 0.5)^\top$ with step size $\alpha = 0.15$. What do you observe?

Solution.

R

```
# Gradient: leverages (b) - at any x at most ONE basin's support is active.
grad <- function(x) {
  d1_sq <- sum((x - c1)^2)
  d2_sq <- as.numeric((x - c2) %*% S2_inv %*% (x - c2))
  if (d1_sq < 1) {
    # Inside c_1 support: grad of rho(||x - c_1||)
    return(as.numeric(-4 * (1 - d1_sq) * (x - c1)))
  } else if (d2_sq < 1) {
    # Inside c_2 support; sign flipped because w_2 = -1
    return(as.numeric(4 * (1 - d2_sq) * (x - c2) %*% S2_inv))
  } else {
    return(c(0, 0)) # dead zone: gradient identically zero
  }
}
```

```

    }
}

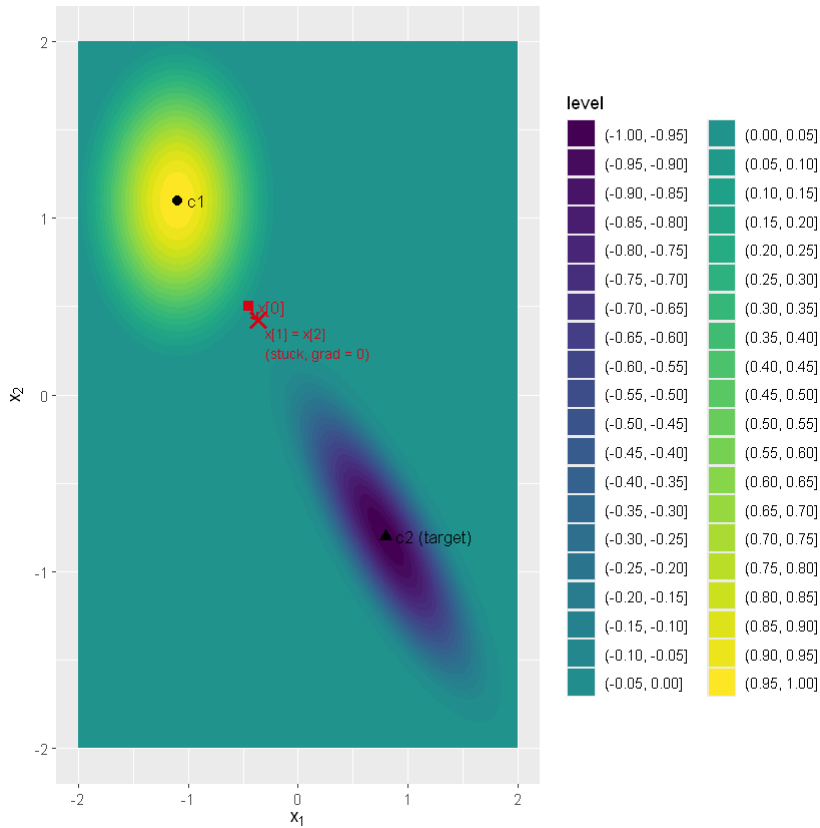
alpha <- 0.15
x0 <- c(-0.45, 0.5)
x1 <- x0 - alpha * grad(x0)
x2 <- x1 - alpha * grad(x1)

cat(sprintf("x[0] = (%+.4f, %+.4f)\n", x0[1], x0[2]))
cat(sprintf("x[1] = (%+.4f, %+.4f)\n", x1[1], x1[2]))
cat(sprintf("x[2] = (%+.4f, %+.4f)\n", x2[1], x2[2]))
cat(sprintf("grad at x[1] = (%+.4f, %+.4f) -- identically zero -> stuck\n",
            grad(x1)[1], grad(x1)[2]))

# Visualize: arrow x[0] -> x[1], then mark the frozen point x[1] = x[2].
cont_plot +
  annotate("segment", x = x0[1], y = x0[2], xend = x1[1], yend = x1[2],
          arrow = arrow(length = unit(0.18, "cm")),
          color = "#D90012", linewidth = 0.7) +
  annotate("point", x = x1[1], y = x1[2], color = "#D90012",
          size = 3, shape = 4, stroke = 1.5) +
  annotate("text", x = x1[1], y = x1[2],
          label = " x[1] = x[2]\n (stuck, grad = 0)",
          hjust = 0, vjust = 1.2, color = "#D90012", size = 3)

x[0] = (-0.4500, +0.5000)
x[1] = (-0.3652, +0.4217)
x[2] = (-0.3652, +0.4217)
grad at x[1] = (+0.0000, +0.0000) -- identically zero -> stuck

```

Python

```
def grad(x):
    """Gradient of f. By (b), at most one basin's support is active at any x."""
    x = np.asarray(x, dtype=float)
    d1_sq = (x - c1) @ (x - c1)
    d2_sq = (x - c2) @ S2_inv @ (x - c2)
    if d1_sq < 1:
        # Inside c_1 support: gradient of +rho(||x - c_1||)
        return -4 * (1 - d1_sq) * (x - c1)
    elif d2_sq < 1:
        # Inside c_2 support: gradient of -rho(||x - c_2||_S2); sign flipped
        return 4 * (1 - d2_sq) * (S2_inv @ (x - c2))
    else:
        return np.zeros(2) # dead zone: gradient identically zero
```

```

alpha = 0.15
x0 = np.array([-0.45, 0.5])
x1 = x0 - alpha * grad(x0)
x2 = x1 - alpha * grad(x1)

print(f"x[0] = ({x0[0]:+.4f}, {x0[1]:+.4f})")
print(f"x[1] = ({x1[0]:+.4f}, {x1[1]:+.4f})")
print(f"x[2] = ({x2[0]:+.4f}, {x2[1]:+.4f})")
g1 = grad(x1)
print(f"grad at x[1] = ({g1[0]:+.4f}, {g1[1]:+.4f})  "
      "-- identically zero -> stuck")

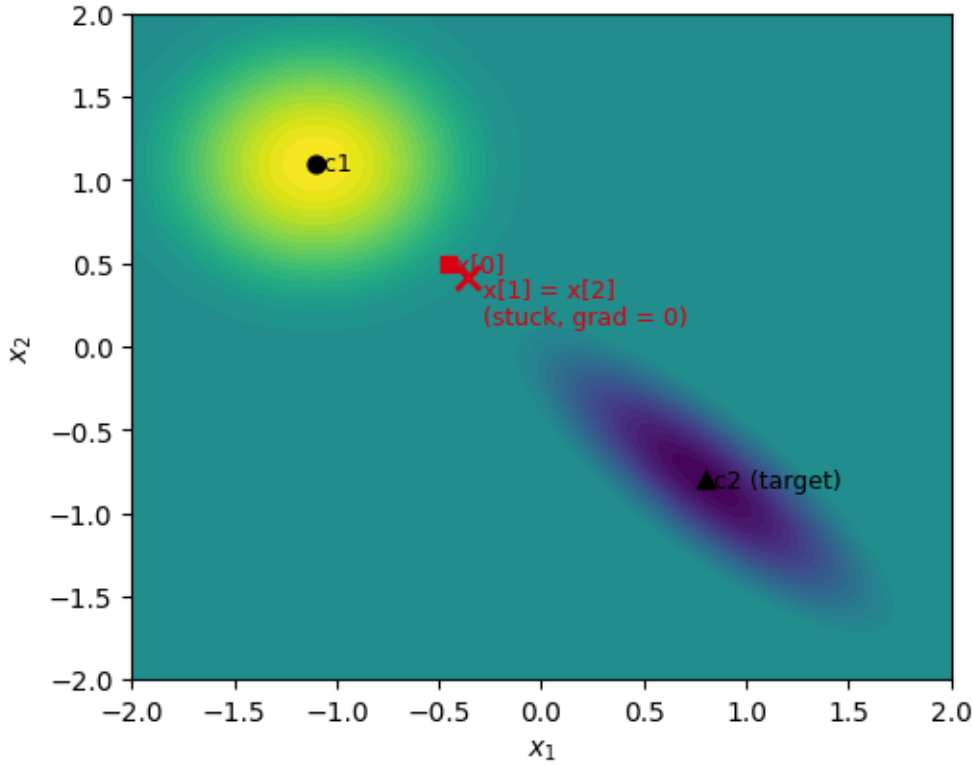
# Visualize the stuck-iterate situation.
fig, ax = plt.subplots(figsize=(5.5, 4.5))
plot_landscape(ax)
ax.annotate("", xy=x1, xytext=x0,
            arrowprops=dict(arrowstyle="->", color="#D90012", lw=1.2))
ax.scatter(*x1, color="#D90012", s=80, marker="x", linewidths=2, zorder=4)
ax.annotate("  x[1] = x[2]\n  (stuck, grad = 0)", x1, color="#D90012",
            fontsize=9, va="top")
plt.show()

```

```

x[0] = (-0.4500, +0.5000)
x[1] = (-0.3652, +0.4217)
x[2] = (-0.3652, +0.4217)
grad at x[1] = (+0.0000, +0.0000)  -- identically zero -> stuck

```



Observation. After the first step the iterate $\mathbf{x}^{[1]}$ lies in the **dead zone** between the two basins. The gradient there is identically zero, so the second step cannot move: $\mathbf{x}^{[2]} = \mathbf{x}^{[1]}$ exactly.

(e) Two GD steps analytically

Confirm (d) with paper-and-pencil arithmetic. Same $\mathbf{x}^{[0]} = (-0.45, 0.5)^\top$, $\alpha = 0.15$.

Solution.

Start with $\mathbf{x}^{[0]} = (-0.45, 0.5)^\top$. Then

$$\|\mathbf{c}_1 - \mathbf{x}^{[0]}\|_2^2 = (-0.65)^2 + (0.6)^2 = 0.4225 + 0.36 = 0.7825 < 1,$$

so $\mathbf{x}^{[0]}$ is inside $\mathbf{c}_1$'s support. The gradient of $f = +\rho(\|\mathbf{x} - \mathbf{c}_1\|)$ here is

$$\nabla f(\mathbf{x}^{[0]}) = -4(1 - \|\mathbf{x}^{[0]} - \mathbf{c}_1\|_2^2) (\mathbf{x}^{[0]} - \mathbf{c}_1) = -4 \cdot 0.2175 \cdot (0.65, -0.6) \approx (-0.5655, 0.5220).$$

Step:

$$\mathbf{x}^{[1]} = \mathbf{x}^{[0]} - 0.15 \cdot \nabla f(\mathbf{x}^{[0]}) = (-0.3652, 0.4217)^\top.$$

Check supports at $\mathbf{x}^{[1]}$:

$$\|\mathbf{c}_1 - \mathbf{x}^{[1]}\|_2^2 \approx 1.0001 > 1, \quad \|\mathbf{c}_2 - \mathbf{x}^{[1]}\|_{S_2}^2 \approx 1.4323 > 1.$$

Both supports missed — $\mathbf{x}^{[1]}$ is in the dead zone, $\nabla f(\mathbf{x}^{[1]}) = \mathbf{0}$, so $\mathbf{x}^{[2]} = \mathbf{x}^{[1]}$.

(f) Gradient descent with momentum

Repeat (d) but with **gradient descent with momentum**. Use velocity initialization $\mathbf{v}^{[0]} = (0.4, -0.4)^\top$, momentum coefficient $\varphi = 0.5$, and run for 15 iterations with step size $\alpha = 0.15$. Find the global minimum.

Why momentum works here. Vanilla GD updates: $\mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} - \alpha \nabla f(\mathbf{x}^{[t]})$. When $\nabla f = \mathbf{0}$ the iterate is frozen.

Momentum adds a velocity that persists across iterations:

$$\mathbf{v}^{[t+1]} = \varphi \mathbf{v}^{[t]} - \alpha \nabla f(\mathbf{x}^{[t]}), \quad \mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} + \mathbf{v}^{[t+1]}.$$

When $\nabla f = \mathbf{0}$, the velocity decays geometrically ($\mathbf{v}^{[t+1]} = \varphi \mathbf{v}^{[t]}$) but does not vanish. With $\varphi = 0.5$ the iterate keeps moving across the dead zone, eventually landing inside $\mathbf{c}_2$'s support — at which point the gradient becomes nonzero and pulls the iterate toward $\mathbf{c}_2$, the true minimum.

Physical analogy. Think of a marble rolling on the surface f . Vanilla GD = marble with no inertia: it stops the instant the floor goes flat. Momentum GD = marble with mass: it carries through flat sections by virtue of its accumulated velocity, and only stops when actually trapped in a basin. The momentum coefficient $\varphi \in (0, 1)$ is friction — closer to 1 = less friction = more inertia.

R

```

alpha <- 0.15
phi <- 0.5
v <- c(0.4, -0.4)
x <- c(-0.45, 0.5)

xs <- x # stack iterates for the trajectory plot
for (i in 1:15) {
  v <- phi * v - alpha * grad(x)
  x <- x + v
  xs <- rbind(xs, x)
}
rownames(xs) <- NULL
trace_df <- data.frame(xs)

cat(sprintf(paste0(
  "x[15] = (%+.4f, %+.4f), f = %+.4f  ",
  "(target c_2 = (%+.4f, %+.4f), f = -1)\n"),
  x[1], x[2], f(x), c2[1], c2[2]))

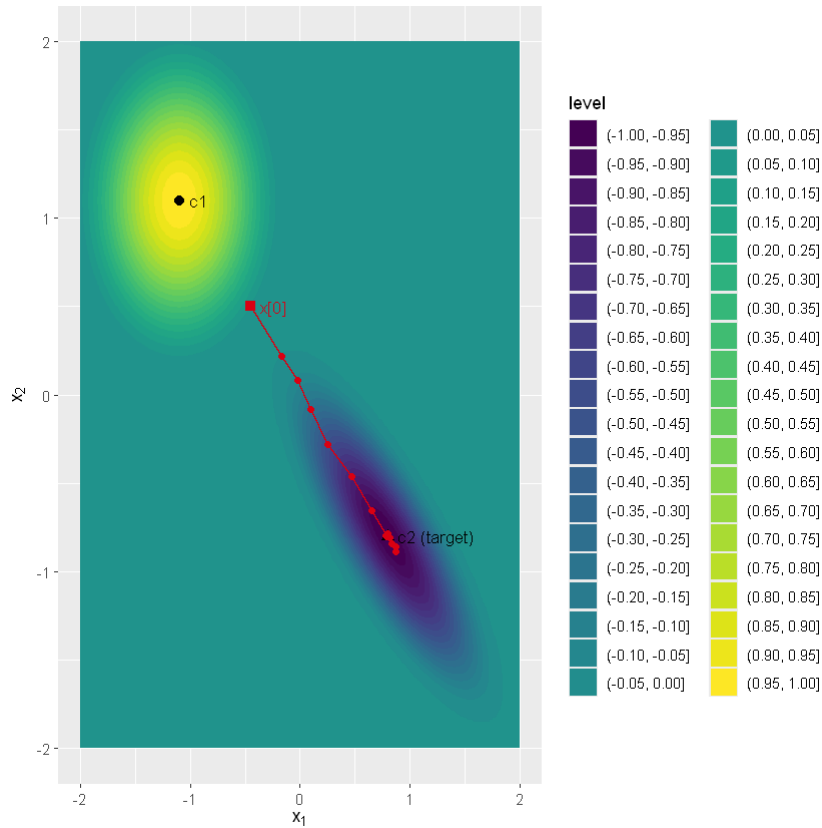
# Trajectory on the contour. cont_plot already has c1, c2, x[0] markers
# from (a); we just overlay the momentum path.
cont_plot +
  geom_path(data = trace_df, aes(x = X1, y = X2),
            color = "#D90012", linewidth = 0.6, inherit.aes = FALSE) +
  geom_point(data = trace_df, aes(x = X1, y = X2),
             color = "#D90012", size = 1.5, inherit.aes = FALSE)

```

```

x[15] = (+0.7831, -0.7951), f = -0.9990 (target c_2 = (+0.8000, -0.8000), f = -1)

```



Python

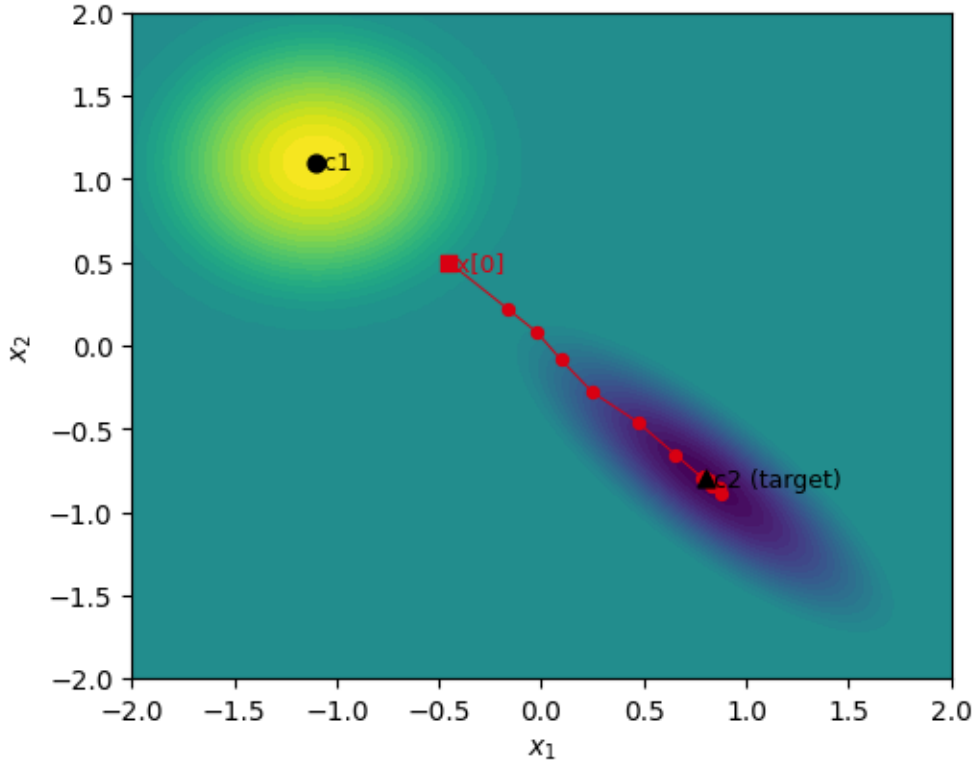
```
alpha = 0.15
phi = 0.5
v = np.array([0.4, -0.4])
x = np.array([-0.45, 0.5])

xs = [x.copy()]
for _ in range(15):
    v = phi * v - alpha * grad(x)
    x = x + v
    xs.append(x.copy())
trace = np.array(xs)

print(f"x[15] = ({x[0]:+.4f}, {x[1]:+.4f}), f = {f(x):+.4f} "
      f"(target c_2 = ({c2[0]:+.4f}, {c2[1]:+.4f}), f = -1)")
```

```
fig, ax = plt.subplots(figsize=(5.5, 4.5))
plot_landscape(ax)
ax.plot(trace[:, 0], trace[:, 1], color="#D90012", linewidth=0.7)
ax.scatter(trace[:, 0], trace[:, 1], color="#D90012", s=18)
plt.show()
```

$x[15] = (+0.7831, -0.7951)$, $f = -0.9990$ (target $c_2 = (+0.8000, -0.8000)$, $f = -1$)



Observation. The trajectory crosses the dead zone purely from inertia (since $\nabla f = \mathbf{0}$ there) and enters c_2 's support after a few iterations. From there the negative-weight bump produces a pull toward c_2 , and the remaining iterations refine to $f \approx -0.999$ — within 0.001 of the true global minimum at c_2 where $f = -1$.